SYSTEM DESIGN INTERVIEW · FULL ARCHITECTURE

Design *Monk*

A complete system design for an AI-powered Accounts Receivable automation platform — requirements, capacity, architecture, components, database, API, scalability, and trade-offs.

01 **Requirements & Scope**
Functional, non-functional, out of scope

02 **Capacity Estimation**
Users, QPS, storage, bandwidth

03 **High-Level Architecture**
The full system — all services and data flows

04 **Core Services Deep Dive**
Contract Parser, Billing Engine, Invoice Service

05 **Collections Agent Architecture**
AI outreach pipeline, state machine, race conditions

06 **Payment Processing**
Stripe integration, webhooks, idempotency

07 **Reporting & Analytics**
Live DSO, aging, multi-source sync

08 **Database Design**
Schema, indexing, sharding strategy

09 **API Design**
REST endpoints, auth, versioning

10 **Scalability & Reliability**
Caching, queues, failover, SLAs

11 **Security & Compliance**
Multi-tenant isolation, SOC 2, PCI

12 **Trade-offs & Decisions**
Every major choice and what you gave up

INTERVIEW FORMAT

This follows the standard big-tech system design interview structure: 5 min requirements → 5 min capacity → 10 min high-level → 15 min deep dives → 10 min scaling → 5 min trade-offs. Use the section headings as your clock. The interviewer will interrupt — that's intentional. Keep driving back to the diagram.

01 Requirements & Scope

5 MINUTES

Start every system design by clarifying scope. State these out loud and wait for the interviewer to agree or redirect before drawing anything.

FUNCTIONAL REQUIREMENTS — WHAT THE SYSTEM MUST DO

- → Ingest contracts (PDF upload) and extract billing terms using AI
- → Generate billing schedules (recurring, milestone, usage-based, hybrid)
- → Auto-generate and send invoices on schedule with no human intervention
- → Run an AI collections agent that follows up on unpaid invoices
- → Accept payments via a public payment portal (no login required)
- → Support partial payments and track running balances
- → Sync with external ERPs (QuickBooks, NetSuite) and Stripe
- → Provide real-time AR reporting: DSO, aging buckets, cash collected
- → Support multiple tenants with data isolation and custom branding
- → Handle contract amendments without breaking existing billing schedules
- → Maintain a full audit log of every payment and status change
- → Upload invoices to enterprise procurement portals (Coupa, Ariba, SAP)

NON-FUNCTIONAL REQUIREMENTS — HOW WELL IT MUST DO IT

AVAILABILITY

99.9% uptime

Payment portal must be highly available. A customer unable to pay = lost cash for our customer. Max 8.7h downtime/year.

DURABILITY

Zero payment loss

Every payment event must be persisted before any downstream action. No at-most-once delivery on the write path.

LATENCY

<300ms p99

Payment page load and submission. Reporting dashboard <1s. Contract parsing is async — no SLA on that step.

CONSISTENCY

Strong on payments

Invoice status, balance, and payment records must be strongly consistent. Reporting can tolerate eventual consistency within seconds.

OUT OF SCOPE — EXPLICITLY SET THESE ASIDE

NOT BUILDING

A payment processor

Monk doesn't process cards — Stripe does. We store payment intent IDs, not card data. PCI compliance delegated entirely to Stripe.

NOT BUILDING

A CRM or ERP

Monk integrates with CRMs and ERPs but doesn't replace them. Customer data is synced in, not mastered in Monk.

NOT BUILDING

A general LLM

The AI layer calls OpenAI/Anthropic APIs. We're designing the orchestration, validation, and retry logic — not the model itself.

SAY IN INTERVIEW

"Before I start drawing, let me nail down scope. The hardest design constraint here is that payments must be strongly consistent — I'm willing to accept eventual consistency on reporting and async on contract parsing, but the payment write path is non-negotiable. That one design decision shapes almost everything downstream."

02 Capacity Estimation

5 MINUTES

Back-of-envelope numbers ground the architecture. State your assumptions explicitly — the interviewer may correct them, and that's fine.

ASSUMPTIONS

USER BASE

10,000 tenants

B2B SaaS scale. Each tenant has 1–20 internal users (finance team). Each tenant has 50–500 customers (payers). ~200K total payer-facing users.

INVOICE VOLUME

5M invoices / month

Average 500 invoices/tenant/month across 10K tenants. 5M invoices = ~1.9 invoices/sec generated. Peak: 10x on billing cycle dates (month-end).

PAYMENT VOLUME

~2.5M payments / month

50% payment rate × 5M invoices = 2.5M payments. ~1 payment/sec average. Peak on invoice due dates: 20–30x spike.

~2	~1	20×	~500
INVOICES/SEC	PAYMENTS/SEC	PEAK MULTIPLIER	API REQ/SEC
avg (5M/month)	avg (2.5M/month)	billing cycle dates	dashboard + portals

STORAGE ESTIMATION

ENTITY	ROW SIZE	ROWS / MONTH	MONTHLY DATA	3-YEAR TOTAL
Contracts	~5 KB (+ PDF in S3)	~100K new/month	~500 MB DB	~18 GB DB + 5 TB S3
Invoices	~2 KB	5M	~10 GB	~360 GB
Payments	~1 KB	2.5M	~2.5 GB	~90 GB
BillingEvents	~500 B	~5M	~2.5 GB	~90 GB
AuditLog	~800 B	~20M events	~16 GB	~576 GB
AR Snapshots	~10 KB	10K tenants (rewritten)	~100 MB	~100 MB (always fresh)
Total DB				**~1.1 TB / year**

BANDWIDTH ESTIMATION

INBOUND

~50 MB/s peak

PDF contract uploads (avg 2MB each), API requests, Stripe webhooks. Contract uploads are the dominant inbound traffic.

OUTBOUND

~200 MB/s peak

Invoice PDFs emailed to customers (avg 200KB), dashboard data, portal renders. Invoice email delivery is the dominant outbound traffic.

LLM API CALLS

~100K calls/day

Contract parsing (1 call/contract), collections agent (1 call/outreach). At 5K input tokens avg, ~500M tokens/day to LLM provider.

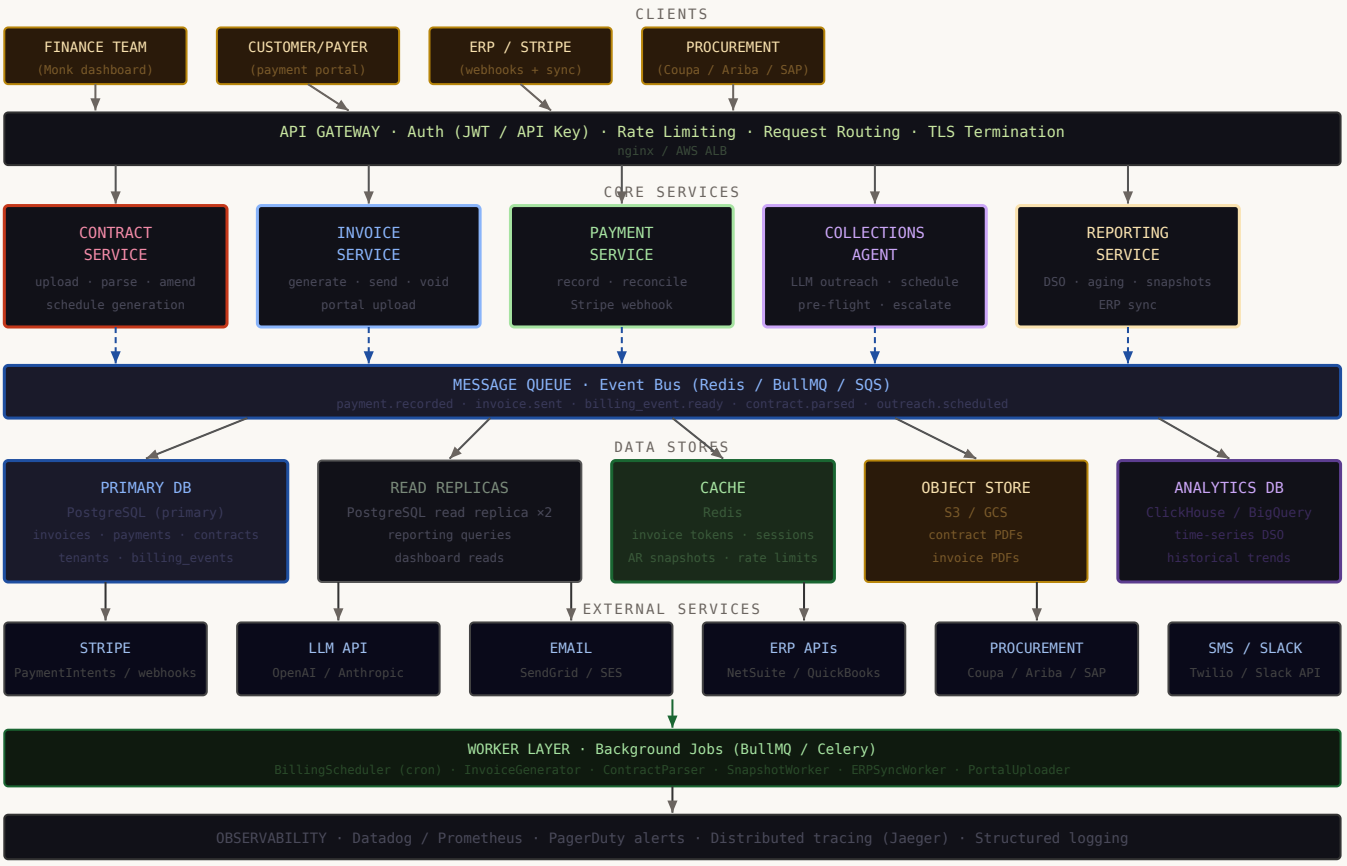
KEY CONCLUSION

"The read:write ratio on invoices is high — customers open payment links many times before paying. Caching is important on the read path. The write path (payments) is low-volume but extremely high-value — correctness beats throughput here. The LLM layer is the most expensive and slowest component — it must always be async and never in the critical path."

03 High-Level Architecture

10 MINUTES

Draw this first. Talk through it before going deep. The interviewer will tell you which component to zoom into.



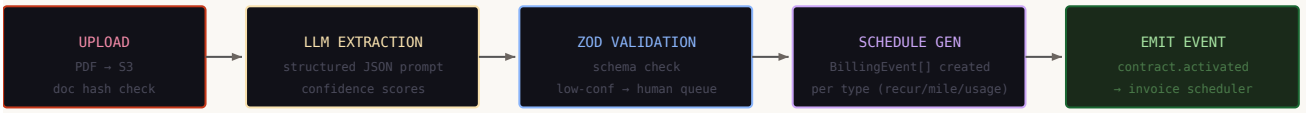
WALK THE INTERVIEWER THROUGH THIS

"I've split this into five layers: clients, API gateway, core microservices, data stores, and external integrations. All inter-service communication happens through the event bus — services never call each other directly. The payment service publishes `payment.recorded`*, the reporting service consumes it and updates the snapshot, the collections agent consumes it and cancels the follow-up job. No service needs to know about the others."*

04 Core Services Deep Dive

15 MINUTES

CONTRACT SERVICE — PDF TO BILLING SCHEDULE



INVOICE SERVICE — BILLINGEVENT TO SENT INVOICE

```
// BillingScheduler: cron job — runs every minute
async function processDueBillingEvents() {
  // find all events that are ready to fire
  const dueEvents = await db.query(
    `SELECT * FROM billing_events
     WHERE status = 'ready'
       AND scheduled_at <= now()
     LIMIT 500`
  );

  // process in parallel with bounded concurrency
  await pMap(dueEvents, generateInvoiceFromEvent, { concurrency: 20 });
}

async function generateInvoiceFromEvent(event: BillingEvent) {
  await db.transaction(async (trx) => {
    // 1. Create invoice — atomic with the event status update
    const invoice = await trx.query(
      `INSERT INTO invoices (tenant_id, customer_id, billing_event_id, ...)
       VALUES ($1, $2, $3, ...) RETURNING *`
    );
    // 2. Mark event as invoiced — idempotency guard
    // WHERE status='ready' prevents double-fire if job runs twice
    const r = await trx.query(
      `UPDATE billing_events SET status='invoiced', invoice_id=$1
       WHERE id=$2 AND status='ready'`,
      [invoice.id, event.id]
    );
    // 3. If rowCount is 0, event was already processed — abort
    if (r.rowCount === 0) throw new Error('Duplicate event — rolling back');
  });
  // 4. Publish to queue — triggers PDF generation + email dispatch
  await queue.publish('invoice.created', { invoiceId: invoice.id });
}
```

BILLING ENGINE — THREE SCHEDULE TYPES

TYPE	SCHEDULE GENERATION	AMOUNT KNOWN?	TRIGGER	CRITICAL EDGE CASE
Recurring	Loop: one event per period from start→end date	**Yes, upfront**	Scheduled date passes	Month-end anchor (Jan 31 → Feb 28)

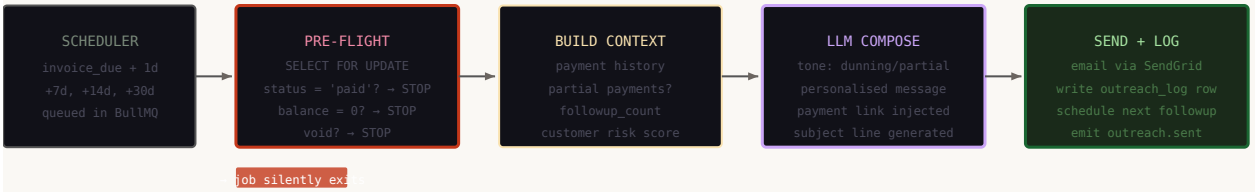
TYPE	SCHEDULE GENERATION	AMOUNT KNOWN?	TRIGGER	CRITICAL EDGE CASE
Milestone	One event per milestone — no scheduledAt set	**Yes, per milestone**	Customer approves milestone	Approval disputed → escalation timer
Usage-based	One event per period — amount null until usage arrives	**No — calculated at fire time**	Period ends + usage data received	Usage data late → hold or estimate
Hybrid	Two parallel schedules: recurring base + usage overage	Mixed	Both triggers independent	Two invoice streams per contract

05 Collections Agent Architecture

AI PIPELINE

The most complex service in the system — an LLM-powered agent that must coordinate with the payment service, respect state transitions, and never email a customer who already paid.

AGENT PIPELINE — FROM OVERDUE INVOICE TO SENT EMAIL



RACE CONDITION — PAYMENT ARRIVES WHILE JOB IS QUEUED

THE PROBLEM (TOCTOU)

Job queued at T=0, paid at T=2, fires at T=5

The agent job is created when the invoice becomes overdue. Two minutes later the customer pays. At T=5 the job fires — without a pre-flight check it sends a dunning email to a customer who already paid. This is the #1 trust-destroying bug in any AR product.

THE FIX — PRE-FLIGHT WITH SELECT FOR UPDATE

Re-read state with a row lock before every action

`SELECT * FROM invoices WHERE id=$1 FOR UPDATE` — this locks the row for the duration of the check. If a payment transaction is concurrently writing to the same row, the pre-flight blocks until it's done, then reads the updated (paid) status and exits cleanly.

ESCALATION STATE MACHINE

DAY OVERDUE	ACTION	TONE	CHANNEL
+1 day	Gentle reminder	Friendly — "just checking in"	Email
+7 days	Follow-up 1	Slightly more direct	Email
+14 days	Follow-up 2	Formal — reference contract terms	Email + SMS
+30 days	Escalation to human	Finance team notified — agent stops	Internal Slack alert
After partial payment	Clock RESETS to day 0	Appreciative — acknowledge payment	Email

06 Payment Processing

CRITICAL PATH

The most correctness-critical service. A bug here means a wrong AR record, a missed dunning email, or a double charge. Every operation on this path must be atomic and idempotent.

PAYMENT FLOW — HAPPY PATH

THE THREE-WAY ATOMIC WRITE — CORE INVARIANT

```
// These three operations are the heart of the payment service
// They must ALWAYS succeed together or ALL roll back — no partial state
await db.transaction(async (trx) => {

  // Guard 1: Lock the invoice row — blocks concurrent payment attempts
  const inv = await trx.query(`SELECT * FROM invoices WHERE id=$1 FOR UPDATE`, [invoiceId]);

  // Guard 2: Validate state — reject stale or invalid attempts
  if (inv.status === 'paid' || inv.status === 'void') throw Error('Invalid state');
  if (amount > inv.balance_due_cents) throw Error('Exceeds balance');

  // Write 1: Insert payment — stripe_intent_id is the idempotency key
  await trx.query(
    `INSERT INTO payments (stripe_intent_id, invoice_id, amount_cents)
     VALUES ($1, $2, $3) ON CONFLICT (stripe_intent_id) DO NOTHING`,
    [stripeIntentId, invoiceId, amount]
  );

  // Write 2: Update balance on invoice
  await trx.query(
    `UPDATE invoices SET amount_paid_cents = amount_paid_cents + $1 WHERE id=$2`,
    [amount, invoiceId]
  );

  // Write 3: Transition status — this is what the agent reads
  const newStatus = amount >= inv.balance_due_cents ? 'paid' : 'partial';
  await trx.query(`UPDATE invoices SET status=$1 WHERE id=$2`, [newStatus, invoiceId]);
});
```

SAY IN INTERVIEW

"I never separate these three writes. They look like they could be optimised into smaller transactions but I intentionally resist that. The moment invoice balance and invoice status can diverge — even for a millisecond — you have a window where the collections agent acts on corrupted data. The transaction is the contract."

07 Reporting & Analytics Service

READ-HEAVY

SNAPSHOT ARCHITECTURE — WRITE ONCE, READ FAST

DSO FORMULA + IMPLEMENTATION

```
// DSO = average days from invoice issued to payment received
// Trailing 90-day window is more useful than all-time average
function computeTrailingDSO(invoices: Invoice[], days = 90): number {
  const cutoff = new Date(Date.now() - days * 86_400_000);

  // only invoices issued within the trailing window that are now paid
  const recentPaid = invoices.filter(
    i => i.status === 'paid' && i.paidAt !== null && i.issuedAt >= cutoff
  );

  if (recentPaid.length === 0) return 0;

  const totalDays = recentPaid.reduce(
    (sum, i) => sum + daysBetween(i.issuedAt, i.paidAt!), 0
  );

  return Math.round(totalDays / recentPaid.length);
}

// Aging bucket classification
function agingBucket(invoice: Invoice): string {
  const daysOverdue = daysBetween(invoice.dueDate, new Date());
  if (daysOverdue <= 0)   return 'current';
  if (daysOverdue <= 30)  return '1_30';
  if (daysOverdue <= 60)  return '31_60';
  if (daysOverdue <= 90)  return '61_90';
  return '90_plus';
  // highest risk — flag for write-off review
}
```

MULTI-SOURCE RECONCILIATION — STRIPE + ERP + MONK

PROBLEM

Three sources, three truths

A payment might exist in Stripe (webhook), in NetSuite (batch sync), and in Monk. They must all reconcile to the same amount and the same invoice.

SOLUTION — CANONICAL INGESTION

Every payment gets (source, externalId)

Unique constraint on `(tenant_id, source, external_id)` prevents the same Stripe payment being ingested twice. Unmatched payments go to a human reconciliation queue.

NIGHTLY RECONCILIATION JOB

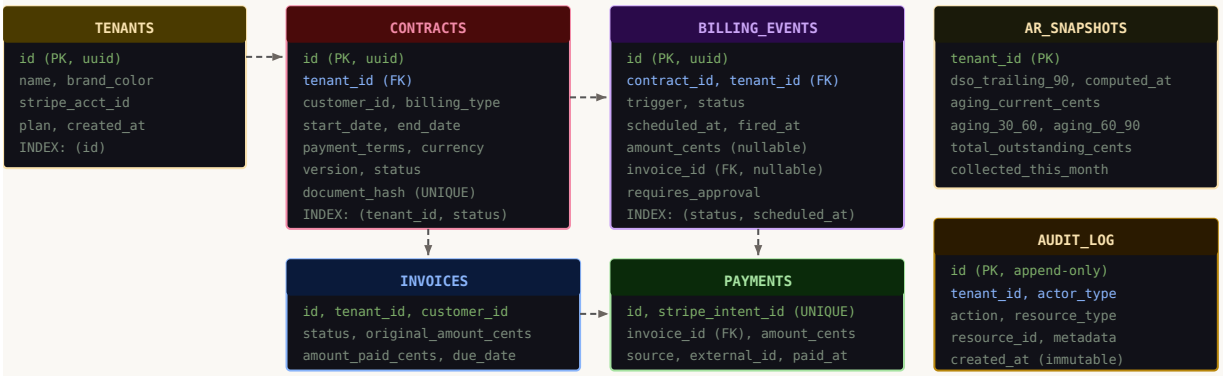
Catch what the events missed

Pull all payments from Stripe API and ERP APIs for the past 24h. Compare against Monk's records. Flag any discrepancy — amount, status, or currency.

08 Database Design

SCHEMA + INDEXING

ENTITY RELATIONSHIP — CORE TABLES



DATABASE CHOICE RATIONALE

STORE	TECHNOLOGY	WHY
Primary	PostgreSQL	ACID transactions are non-negotiable on the payment write path. Joins across invoices, payments, allocations. Row-level locking (SELECT FOR UPDATE). Full-text search on contracts.
Cache	Redis	Invoice token cache (TTL = token expiry). AR snapshot cache. BullMQ job queue. Rate limiting counters. Session store.
Object store	S3 / GCS	Contract PDFs (avg 2MB). Invoice PDFs. Signed URLs with expiry for customer access.
Analytics	ClickHouse	Time-series DSO trend data. Historical aging. High-cardinality group-by queries on millions of invoice rows. Columnar compression.
Sharding strategy	By tenant_id	Tenant is the natural isolation boundary. All queries include tenant_id. At scale, partition PostgreSQL by tenant_id range or hash. Large tenants get dedicated DB instances.

09 API Design

REST + AUTH

AUTHENTICATION STRATEGY

FINANCE TEAM (INTERNAL)

Session JWT via Clerk/Auth0

Standard OAuth 2.0 / OIDC login. Short-lived access token (15 min) + refresh token. Tenant_id embedded in token — row-level scope on every DB query.

CUSTOMER (PUBLIC PORTAL)

Signed payment token

No account required. One-time JWT in URL signed with HMAC-SHA256. Contains invoice_id + tenant_id. 30-day expiry. Revoked via jti stored in DB.

ERP / WEBHOOKS (MACHINE)

API Key + HMAC signature

Static API key per tenant for ERP integration. Stripe webhooks validated via Stripe-Signature header. Keys stored hashed — never in plaintext.

CORE REST ENDPOINTS

```
// ── CONTRACT SERVICE ──────────────────────────
POST   /api/v1/contracts              // upload PDF — triggers async parse job
GET    /api/v1/contracts/:id          // get contract + extracted terms + confidence
PATCH  /api/v1/contracts/:id/amend   // apply amendment — cancels future events
GET    /api/v1/contracts/:id/schedule // list billing events for this contract

// ── INVOICE SERVICE ───────────────────────────
GET    /api/v1/invoices               // list — paginated, filterable by status/date
GET    /api/v1/invoices/:id           // single invoice with line items
POST   /api/v1/invoices/:id/void     // void invoice — creates audit log entry
POST   /api/v1/invoices/:id/resend   // resend payment link — rotates JWT

// ── PAYMENT PORTAL (PUBLIC — JWT token auth) ──
GET    /pay/:token                    // render payment page — no login required
POST   /pay/:token/intent             // create Stripe PaymentIntent — returns client_secret
GET    /pay/:token/status             // poll for payment confirmation

// ── WEBHOOK ENDPOINTS ─────────────────────────
POST   /webhooks/stripe               // Stripe payment events — HMAC verified
POST   /webhooks/erp/:provider       // ERP payment sync — API key auth

// ── REPORTING ─────────────────────────────────
GET    /api/v1/reporting/snapshot     // current DSO, aging, cash — single row read
GET    /api/v1/reporting/dso-trend   // time-series DSO over N days
GET    /api/v1/reporting/aging       // full aging report with customer breakdown
```

API VERSIONING + ERROR SHAPE

```
// All endpoints prefixed /api/v1/ — breaking changes go to v2
// Non-breaking additions (new fields) are fine in the same version

// Standard error envelope — consistent across all endpoints
{
  "error": {
    "code":    "INVOICE_ALREADY_PAID",     // machine-readable
    "message": "Invoice is already paid",  // human-readable
    "details": { "invoice_id": "...", "status": "paid" },
    "request_id": "req_01abc..."           // for support tracing
```

```
  }
}
```

10 Scalability & Reliability

10 MINUTES

SCALING EACH LAYER

LAYER	BOTTLENECK	SOLUTION	AT WHAT SCALE
API Gateway	Throughput	Horizontal scale behind ALB. Rate limiting per tenant_id (Redis sliding window). CDN for static assets.	Day 1
Payment Service	DB lock contention	SELECT FOR UPDATE is fine at B2B scale. At 1000 concurrent payments/sec, move to queue-per-invoice with partitioned consumers.	100K tenants
Invoice Generator	Batch on billing dates	Workers pool with concurrency=20. Priority queue for overdue. Spread billing dates with per-tenant offset.	10M invoices/month
LLM Contract Parser	API rate limits + cost	Always async — never synchronous. Per-tenant quota. Retry with exponential backoff. Cache parsed results by document hash.	Day 1
Collections Agent	LLM latency	Pre-generate email drafts 1h before scheduled send. Human-review queue for low-confidence messages.	1M outreaches/month
Read DB	Dashboard queries	Read replicas for reporting. AR snapshots eliminate expensive aggregate queries. ClickHouse for trend data.	5M invoices
DB Write	Row count	Partition invoices by tenant_id. Archive paid invoices older than 2 years to cold storage. Logical replication.	500M rows

RELIABILITY — HANDLING FAILURES

STRIPE WEBHOOK MISSED

Nightly reconciliation job

Pull all payments from Stripe API for the past 24h. Compare against Monk's payment records. Any Stripe payment not in Monk → ingest and reconcile. This is the safety net for dropped webhooks.

LLM API DOWN

Async queue + retry

Contract parsing is always async — user uploads PDF, gets a "processing" status immediately. If LLM is down, job sits in queue with exponential backoff. SLA: parse within 4 hours, not real-time.

SNAPSHOT WORKER CRASH

Atomic swap — old row valid until new commits

Snapshot update rewrites the entire row in one transaction. Old snapshot remains valid and readable until the new one commits. No partial state. Nightly full recompute is the final fallback.

DB PRIMARY FAILURE

Automatic failover to replica

PostgreSQL streaming replication with auto-failover (Patroni / AWS RDS Multi-AZ). RPO < 1 min. RTO < 30s. Payment service uses write endpoint URL — transparent to application on failover.

CACHING STRATEGY

INVOICE TOKEN

Redis TTL = token expiry

Cache token → invoice mapping. Avoids DB hit on every payment page load. Invalidated on void. Cache miss → DB lookup + re-cache.

AR SNAPSHOT

Redis TTL = 60s

Dashboard reads from Redis first. Snapshot worker writes to both Postgres and Redis. Dashboard load = one Redis GET, not a DB scan.

TENANT CONFIG

Redis TTL = 5 min

Logo URL, brand color, feature flags. Loaded on every payment page render. Cache means no DB roundtrip for static branding data per request.

11 Security & Compliance

SOC 2 + PCI

MULTI-TENANT ISOLATION

```
// Every query MUST include tenant_id — enforced at the ORM/query layer
// Never trust tenant_id from user input — always pull it from the validated JWT
const tenantId = req.user.tenantId;
// from the decoded + verified JWT — not from req.body

// All DB queries are tenant-scoped by default
const invoice = await db.query(
  `SELECT * FROM invoices WHERE id = $1 AND tenant_id = $2`,
  [invoiceId, tenantId]
  // without tenant_id, a guessed ID leaks another tenant's data
);

// Consider PostgreSQL Row Level Security as a defence-in-depth layer
// CREATE POLICY tenant_isolation ON invoices
// USING (tenant_id = current_setting('app.tenant_id')::uuid)
```

SECURITY CONTROLS BY LAYER

LAYER	CONTROL	WHY
Transport	TLS 1.3 everywhere, HSTS, no HTTP	Financial data in transit — non-negotiable
Auth	JWT with short expiry (15 min). Refresh token rotation.	Stolen access token expires fast. Refresh token theft detectable.
Payment tokens	HMAC-SHA256 signed, stored hashed, revocable via jti	Even if URL is intercepted, can't forge a different invoice_id
Card data	Never touches Monk servers. Stripe Elements handles it.	PCI DSS out of scope — Stripe is the PCI-compliant party
Secrets	AWS Secrets Manager / Vault. Never in env vars or code.	Stripe keys, JWT secrets, LLM API keys — rotated quarterly
DB encryption	Encryption at rest (AES-256). Column encryption for PII.	SOC 2 Type II requirement. Contact email encrypted at column level.
Audit log	Append-only table. No UPDATE or DELETE allowed.	Tamper-evident record of every financial event. Required for SOC 2.
Rate limiting	Per-tenant sliding window at API gateway (Redis)	Prevent one tenant from DoS-ing shared infrastructure

SOC 2 REQUIREMENTS THAT DRIVE DESIGN

"Three SOC 2 controls directly shaped this architecture: (1) the audit log must be append-only and immutable — that's why it's a separate table with no UPDATE privileges; (2) multi-tenant data isolation must be provable — that's why tenant_id is in every query and we use Postgres RLS as a backstop; (3) every access to financial data must be logged — that's why the audit log captures actor, action, and before/after state for every payment event."

12 Trade-offs & Key Decisions

5 MINUTES

Every design decision is a trade-off. State yours explicitly — it shows senior engineering judgment.

DECISION 1 — MICROSERVICES VS MONOLITH

CHOSE: SERVICE-ORIENTED (NOT FULL MICROSERVICES)

- → Separate deployments per domain (contract, invoice, payment, reporting)
- → Independent scaling — invoice generator scales separately from reporting
- → Failure isolation — LLM service down doesn't take down payment processing
- → Team ownership boundaries align with service boundaries

GAVE UP

- → Cross-service transactions are now impossible — use eventual consistency + events
- → Higher operational overhead — more services to deploy and monitor
- → Network latency on inter-service calls — mitigated by async event bus
- → Would have started with a monolith and split only when needed

DECISION 2 — STRONG CONSISTENCY ON PAYMENTS VS EVENTUAL CONSISTENCY

CHOSE: STRONG CONSISTENCY FOR PAYMENT WRITES

- → Three-way atomic transaction — payment, balance, status in one commit
- → SELECT FOR UPDATE for concurrent payment prevention
- → Synchronous webhook processing before responding 200 to Stripe
- → Reporting is eventually consistent (seconds) — acceptable trade-off

GAVE UP

- → Higher DB latency — transactions hold locks longer
- → Lower throughput on the payment endpoint — but B2B scale makes this fine
- → Cannot use a distributed DB (DynamoDB, Cassandra) for payments — no transactions
- → Write amplification — one payment = 3 DB writes + event publish

DECISION 3 — SNAPSHOT PATTERN FOR REPORTING VS LIVE QUERIES

CHOSE: PRE-COMPUTED SNAPSHOTS

- → Dashboard reads are O(1) — single PK lookup on ar_snapshots
- → No reporting query can slow down the write path
- → Snapshots are updated async via event bus — payment processing is unaffected
- → Nightly recompute catches any missed events

GAVE UP

- → Reporting is eventually consistent — dashboard may lag by seconds
- → Snapshot worker is a critical background job — must be monitored
- → Custom date-range queries (not in snapshot) still need DB queries
- → Snapshot schema changes require a migration + full recompute

DECISION 4 — LLM FOR COLLECTIONS VS RULE-BASED TEMPLATES

CHOSE: LLM-GENERATED OUTREACH	GAVE UP
→ 24% higher response rate vs templated emails (Monk's own data) → Personalised to customer payment history, amount, relationship → Can adapt tone — appreciative after partial payment, formal for 90+ days → Works across different contexts without manual template management	→ Higher cost per outreach — LLM API calls are not free → Latency — LLM adds 2–5s to agent pipeline — must be async → Non-determinism — output requires review queue for edge cases → LLM API outage = agent pipeline stalls — must queue and retry

THE ONE SENTENCE PER TRADE-OFF — SAY THESE OUT LOUD

* Services not monolith — because LLM failure ≠ payment failure	* PostgreSQL not DynamoDB — ACID on payments, full stop	* Snapshots not live queries — O(1) reads, async updates
* Event bus not direct calls — services don't know about each other	* LLM not templates — 24% better response rate, pay the latency cost	* Async LLM — never on critical payment path, always queued
* Tenant_id in every query — isolation by convention + RLS backstop	* Stripe not own processor — PCI scope delegation is worth the fee	* Start monolith in reality — split only when teams need independence